

FRONTEND TECHNICAL ASSESSMENT

Product Gallery & Variant Experience

A focused 6-hour implementation challenge for candidates applying for a Next.js + React Native / Expo frontend role.

MAXIMUM IMPLEMENTATION TIME

6 hours

REQUIRED PLATFORMS

Next.js Web + React Native / Expo

PRODUCTS

12 products

LIVE DEMO

Mandatory

LANGUAGE

TypeScript only

Goal

Demonstrate real frontend engineering ability: design-system discipline, reusable component structure, responsive implementation, variant state management, image interaction, gesture handling, and production-quality execution.

This assessment intentionally excludes backend work so the full time budget can be spent on frontend quality.

1. Scope

Build the same product browsing experience for **Next.js Web** and **React Native / Expo**. Product data and image assets are local/mock data. No backend implementation is required.

REQUIREMENT	DESKTOP WEB	MOBILE WEB	REACT NATIVE APP
Product count	12	12	12
Grid	6 products per row	2 products per row	2 products per row
Product image frame	3.5:6	3.5:6	3.5:6
Color swatches on card	Required	Required	Required
Image viewer	Required	Required	Required
Swipe between images	Drag / controls acceptable	Finger swipe required	Finger swipe required
Pinch-to-zoom + pan	Touch support or equivalent zoom control	Required	Required

2. Product Card Requirements

Visual structure

Product image, favorite icon, product name, current price, old price / discount when provided, and visible color swatches.

Image ratio

The product image container must maintain a fixed

3.5:6 (width:height)

portrait ratio. Different source image sizes must not change the card height or cause layout shift.

Color selection

Clicking/tapping a color swatch on the listing card must immediately switch the main product image to that color variant without reloading the page/screen.

Persistent card state

The selected color must remain selected while the user opens and closes the image viewer for that product.

Suggested local data shape

```
{
  id: 1,
  name: "Product name",
  price: 129,
  oldPrice: 169,
  variants: [
    {
      color: "Black",
      colorCode: "#000000",
      images: ["black-1.webp", "black-2.webp", "black-3.webp"]
    }
  ]
}
```

Each color variant may contain a different number of images. The implementation must handle this safely.

3. Image Viewer / Gallery

- Opening a product image must open an image viewer for the **currently selected color only**.
- The image presentation must preserve the same 3.5:6 product-image visual frame inside the viewer.
- Mobile Web and React Native must support natural **left/right finger swiping** between images.
- Mobile Web and React Native must support real **pinch-to-zoom** and **pan/drag** while zoomed.
- A current-image indicator (for example dots or image count) must be visible.
- The viewer must close cleanly and restore the product state.

Default image zoom: The product image starts at the default zoom/crop level defined by the supplied design. The user must be able to zoom in and zoom out relative to that default level. The initial position must be centered and visually consistent.

Variant change while viewer is open

If the user changes the color while the gallery is open:

- Load the first image of the newly selected color.
- Reset zoom to the default zoom level.
- Reset pan position to center.
- Do not reload the page/app and do not show images from the previous color.

4. Design System Compliance

The supplied Figma / design specification and design tokens are the source of truth. The implementation will be evaluated for visual accuracy as well as code quality.

- Follow supplied typography, spacing, colors, radii, shadows, icon treatment, and image behavior.
- Do not invent alternative visual styles when a design-system value is provided.
- Avoid unnecessary hardcoded visual values when an equivalent design token exists.
- Use reusable components. At minimum, the implementation should demonstrate sensible reuse for ProductCard, color swatches, price display, image viewer, and loading/fallback behavior.
- The UI must remain stable while images are loading. No visible layout jumping.

5. Technical Requirements

Web

Next.js, App Router,

TypeScript mandatory

. Plain JavaScript implementations will not be accepted. Use appropriate Next.js image optimization and responsive behavior.

Mobile App

React Native, Expo,

TypeScript mandatory

. Plain JavaScript implementations will not be accepted. Gesture/image libraries are allowed, but the candidate must be able to explain the choice and implementation.

State

Keep variant, selected image, gallery, and zoom behavior predictable and isolated. Avoid unnecessary global state or overengineering.

Data

Use local/mock product data only. No backend or database is required.

TypeScript requirement: Both projects must use TypeScript with meaningful types for product data, variants, component props, state, and interaction logic. Excessive use of any, disabling TypeScript checks, or bypassing type safety will negatively affect the evaluation.

6. Required States

- Image loading state / placeholder.
- Broken-image fallback.
- Selected and unselected color states.
- Gallery open / close behavior.
- Current image index / indicator.
- Zoom reset when changing variant.

7. Time Limit and AI Policy

Maximum implementation time: 6 hours.

Stop when the time limit is reached, even if some items remain incomplete. Clearly list incomplete items and trade-offs in the README.

AI-assisted development is not permitted for this assessment. Do not use ChatGPT, Claude, Gemini, GitHub Copilot, Cursor generation, Codex, Windsurf, or similar generative coding tools. Official documentation, package documentation, Stack Overflow, and normal search engines are allowed.

Prebuilt full storefront templates or copied product-gallery implementations are not allowed. Open-source libraries for gestures, image viewing, and standard UI primitives are allowed when declared in the README.

8. Submission Requirements

- Git repository containing **/web** and **/mobile** implementations, or an equally clear project structure.
- README with setup instructions, architecture decisions, libraries used, completed items, known limitations, trade-offs, and actual time spent.
- Meaningful Git history. Aim for at least **5 meaningful commits**.
- **Public live Web demo URL** (for example Vercel or Netlify).
- **Runnable live mobile demo** using Expo Go / EAS Preview / installable preview build. A recorded video alone is not accepted as the mobile demo.

Live demo is mandatory. The submitted web URL and mobile preview must run on our device without requiring access to the candidate's local machine.

9. Live Review

After submission, shortlisted candidates will join a short live technical review. The candidate must:

- Run and demonstrate both Web and React Native versions live.
- Explain component structure, variant state, image viewer behavior, gesture handling, and key technical trade-offs.
- Explain any third-party libraries used.
- Be prepared for a small live change to the submitted code.

Example live change: add a new product color with a different image count and ensure that switching to it while zoomed resets the gallery to image 1, default zoom, and centered pan position.

10. Evaluation

AREA	WEIGHT
Design accuracy and design-system compliance	20%
Next.js implementation	15%
React Native / Expo implementation	15%
Image gallery and swipe quality	15%
Pinch zoom, zoom-out, and pan quality	15%
Variant/color state architecture	10%
Responsive behavior and performance	5%
Code structure, Git discipline, and README	5%

Hard-fail conditions

- **No functional live demo.**
- **Plain JavaScript submission instead of TypeScript for either required project.**
- **React Native implementation is replaced by a WebView/web-only solution.**
- **No real pinch-to-zoom on the mobile app.**
- **Changing color does not correctly change the card and gallery image set.**
- **Major departure from the supplied design system.**
- **Candidate cannot explain or safely modify the submitted code during live review.**